

THE DIDACTIC SOC PLATFORM⁶

The Didactic SoC platform¹ is the baseline SoC template for Edu4Chip. It offers a microcontroller-scale open-source platform for education. The SoC was developed based on principles of simplicity, extendability, reusability, and ease of integration.

2.1 Specification

The Didactic SoC architecture was designed around two distinct functional sections: the management section, also called the staff section, and the student sections in which student subsystems are integrated. These sections are highlighted in Fig. 2.1: the management section on the left of the figure is connected through a global interconnect network (ICN) to the student subsystems (SS) on the right of the figure. Student sections integrate custom subsystems with a well-defined interface that can be individually selected. The generic interface of the subsystems is described in *Student subsystems*.

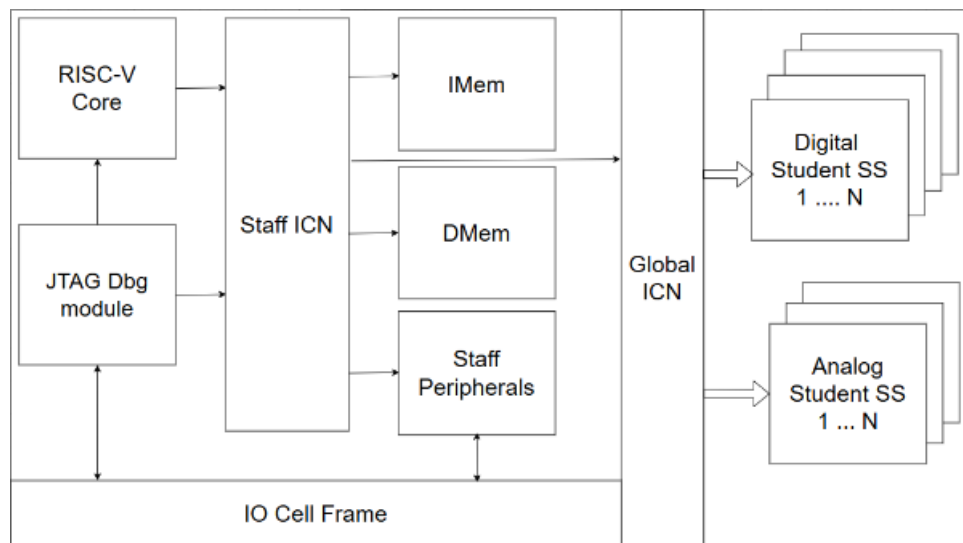


Fig. 2.1: Block diagram of the Didactic SoC architecture.

The staff section of the chip offers:

- a RISC-V Ibex⁴ processor core that implements the RV32IMC instruction set architecture
- a debug module accessible over JTAG from the PULP open source project³ which is compliant with the RISC-V Debug Specification

⁶ AI tools in combination with careful proofreading and rewriting were partially used to support the writing of this section.

¹ <https://github.com/Edu4Chip/Didactic-SoC>

⁴ <https://github.com/lowRISC/ibex>

³ <https://github.com/pulp-platform>

- one 16 KiB SRAM instruction memory (IMEM) and one 16 KiB SRAM data memory (DMEM)
- a set of external peripherals (UART, SPI) from the PULP project [Page 3, 3](#) to communicate with the SoC
- a peripheral interface to up to five freely customisable submodules
- dedicated circuitry that controls subsystem reset and clock enable on one hand, and configures the connection of the subsystems to the pads of the SoC on the other

The initial specification targets the GF 22 nm ASIC technology from Europractice with a maximum frequency of 100 MHz and a maximum area of 2.5 mm² for both the staff section and the student subsystems. The system provides 16 GPIO connections and SPI and UART peripherals. The core and IO voltages are determined by the technology node: 0.8 V for the core and 1.2 V / 1.5 V / 1.8 V for the IOs, which requires the test PCB to use level shifters compatible with 3.3 V peripheral modules.

The above specifications are summarized in [Table 2.1](#).

Table 2.1: Didactic SoC technical specifications

Parameter	Value
Technology node	GF 22nm FDX (Europractice)
Maximum frequency	100 MHz
Maximum area	2.5 mm ² (staff section + student subsystems)
Core voltage	0.8 V
IO voltage	1.2 V / 1.5 V / 1.8 V
CPU core	RISC-V Ibex, RV32IMC ISA
Instruction memory	16 KiB SRAM
Data memory	16 KiB SRAM
Debug interface	JTAG (RISC-V Debug Specification)
GPIO	16 pins
Serial interfaces	UART, SPI
Student subsystem slots	Up to 5
Student subsystem bus	APB

2.1.1 Pinout

The students IPs and peripherals of the Didactic SoC require 32 IO pins. On top of these, analog student subsystems can have their own IO pins. The digital IO connections are routed through a centralized module to handle instantiation of technology specific cells. The rest of the SoC IO area is filled with power and ground connections during the physical implementation stage.

2.1.2 IP-XACT model

IP-XACT is an IP exchange format implemented using XML. It is specified in the IEEE-1685 standard and managed by Accelera. This format enables designers to capture structural descriptions, memory content and information pertaining to design files and documentation. The XML description allows in turn to describe design information in a standardized, tool-friendly format. Didactic platform uses the Kactus2 Graphical User Interface to represent and manipulate the IP-XACT description of the SoC.

2.1.3 Dependency management

This SoC depends on open-source implementations provided by developers at various repositories, primarily on GitHub under the OpenHW Group² and PULP Platform^{Page 3, 3}. These reused hardware modules reference specific commits or tags to track the version of each dependency that corresponds to a given release of the Didactic SoC. Both the Didactic SoC and PULP Platform modules depend on the same bus-specific modules, which leads to the traditional dependency management challenge in hardware development.

This challenge can be addressed in various ways, such as monolithic repositories that copy dependencies inline, or through the use of git submodules that always point to a specific version of a repository. The former makes it difficult to update modules while crediting their authors, and the latter leads to inflexible repository structures. During module development this is not usually a roadblock, but whenever an additional developer joins the project, it tends to cause issues when building consistent development environments across partners.

To facilitate managing repositories at multiple levels in hardware projects, the PULP Platform developed a tool called Bender⁵. It uses a YAML configuration file to represent project dependencies and enforce a specific project structure for submodules. It is then able to resolve dependency chains of all submodules that use the same structure. This mechanism allows the top-level project to easily fetch third-party IPs and manage the versions of the submodules.

2.2 Memory and bus architecture

2.2.1 Bus architecture

The Didactic SoC bus architecture was designed to be both topology- and type-agnostic. The only requirement is that the bus type must be addressable to expose peripherals and subsystems through a unified memory map. IP-XACT modules were used to keep track of bus types and memory-mapped components.

The SoC was first developed using the AXI4LITE ARM AMBA protocol for high-speed bus communication, before switching to the open-source OBI protocol standardised by the OpenHW Group for the taped-out version of the SoC. An OBI-to-APB bridge is used to simplify the interface of the student subsystems.

The SoC bus is currently implemented as two hierarchically separated layers of fully connected crossbars in the global interconnect network (ICN). The first layer routes connections to all submodules of the staff section, and the second layer manages CPU access to the student subsystems.

2.2.2 Memory organisation

System memory map

The memory address layout of the soC is given in Table 2.2. The SoC memory is partitioned across modules, each occupying a fixed address range in the 32-bit address range. The address space was not manually packed.

² <https://github.com/openhwgroup>

⁵ <https://github.com/pulp-platform/bender>

Table 2.2: System memory layout

System	Base Address
Instruction memory	'h01000000
Data memory	'h01010000
Debug module	'h01020000
Peripherals	'h01030000
Control registers	'h01040000
Student subsystems	'h01050000

Peripheral memory map

The memory address layout of the SoC peripherals is given in [Table 2.3](#).

Table 2.3: Peripheral memory layout

Peripheral	Base Address
GPIO	'h01030000
UART	'h01030100
SPI	'h01030200

Student subsystems memory map

The memory address layout of the available slots for student subsystems is given in [Table 2.4](#).

Table 2.4: Student subsystems memory layout

Student subsystem	Base Address
SS0	'h01050000
SS1	'h01060000
SS2	'h01070000
SS3	'h01080000
SS4	'h01090000

2.3 Debug Support

Debug access is one of the standard access modes on a SoC, as an alternative to independent or guided boot. In the Didactic SoC, it is used as the sole boot option for simplicity.

2.3.1 JTAG debug interface

JTAG is an IEEE standard that defines SoC access through a dedicated serial interface. The debug module connected to this interface accepts commands from a host PC. This allows standard tools such as GDB to control the SoC.

2.3.2 Device programming

Device programming is performed in C. The repository includes generic helper functions in header files to support rapid development of test and application code.

2.3.3 Boot sequence

The first iteration of the SoC does not have an independent boot capability. It is fully externally controlled via JTAG. The boot sequence is as follows:

1. SoC is connected to host PC via USB
2. SoC is lifted from reset
3. OpenOCD takes control of debug module via JTAG through ftdi-module
- A) Manual operation through commands:
 4. Connection is established through RISC-V gdb
 5. RISC-V gdb is used to control SoC with direct commands
- B) Programming flow:
 4. Baremetal program is compiled using RISC-V toolchain
 5. RISC-V gdb is used to control the SoC and preload instruction memory
 6. RISC-V gdb lifts core from halt
 7. Core reads instruction memory and starts program execution

Later iterations of the Didactic SoC may include an independent boot mode, in which the core would start from a program stored in ROM.

2.4 Controller

The controller block (`SS_Ctrl_reg_array`) is the central control register bank mapped at base address `0x01040000`. It manages CPU fetch enable, subsystem reset and clock gating, PMOD routing, and IO cell pad configuration. The complete register map is given in [Table 2.5](#).

Table 2.5: Controller register map (base address **0x01040000**)

Register name	Offset	Description
<code>fetch_en</code>	0x000	Ibex CPU instruction-fetch enable
<code>ss_rst</code>	0x004	Subsystem and ICN reset control (active-high)
<code>icn_ss_ctrl</code>	0x008	ICN subsystem control word
<code>ss_0_ctrl</code>	0x00C	Subsystem 0 clock and IRQ control
<code>ss_1_ctrl</code>	0x010	Subsystem 1 clock and IRQ control
<code>ss_2_ctrl</code>	0x014	Subsystem 2 clock and IRQ control
<code>ss_3_ctrl</code>	0x018	Subsystem 3 clock and IRQ control
<code>ss_4_ctrl</code>	0x01C	Subsystem 4 clock and IRQ control
<code>ss_ctrl_reserved_1</code>	0x020	Reserved for a future subsystem slot
<code>pmod_sel</code>	0x024	PMOD connector host-selection mux
<code>io_cell_cfg_0</code>	0x028	IO cell configuration for the UART RX pad
<code>io_cell_cfg_1</code>	0x02C	IO cell configuration for the UART TX pad
<code>io_cell_cfg_2</code>	0x030	IO cell configuration for the SPI SCK pad
<code>io_cell_cfg_3</code>	0x034	IO cell configuration for the SPI CSN0 pad
<code>io_cell_cfg_4</code>	0x038	IO cell configuration for the SPI CSN1 pad
<code>io_cell_cfg_5</code>	0x03C	IO cell configuration for the SPI DATA0 pad
<code>io_cell_cfg_6</code>	0x040	IO cell configuration for the SPI DATA1 pad
<code>io_cell_cfg_7</code>	0x044	IO cell configuration for the SPI DATA2 pad
<code>io_cell_cfg_8</code>	0x048	IO cell configuration for the SPI DATA3 pad
<code>io_cell_cfg_9 – io_cell_cfg_24</code>	0x04C – 0x088	IO cell configuration for GPIO[0]–GPIO[15] pads
<code>return_reg_0</code>	0x100	Post-main idle-loop instruction word 0
<code>return_reg_1</code>	0x104	Post-main idle-loop instruction word 1
<code>boot_reg_0</code>	0x180	Boot idle-loop instruction word 0
<code>boot_reg_1</code>	0x184	Boot idle-loop instruction word 1

The register fields are described below.

`fetch_en` (offset **0x000**)

- `fetch_en_reg` [3:0] — Ibex instruction-fetch enable. Reset value **0x5**. Write a non-zero value to enable instruction fetch; write **0x0** to halt the CPU.

`ss_rst` (offset **0x004**)

- `icn_rst` [0] — ICN reset. Write 1 to hold the interconnect in reset; write **0** to release it.
- `ss_0_rst` [1] — Subsystem 0 reset. Write 1 to hold SS0 in reset; write **0** to release.
- `ss_1_rst` [2] — Subsystem 1 reset. Write 1 to hold SS1 in reset.
- `ss_2_rst` [3] — Subsystem 2 reset. Write 1 to hold SS2 in reset.
- `ss_3_rst` [4] — Subsystem 3 reset. Write 1 to hold SS3 in reset.
- [31:5] — Reserved, write **0**.

`icn_ss_ctrl` (offset **0x008**)

- `icn_ctrl` [30:0] — Control word forwarded to the ICN subsystem port. Currently unused by the ICN logic.

`ss_N_ctrl` (offsets **0x00C–0x01C**, one register per subsystem $N = 0\text{--}4$)

- `ssN_clk_en` [0] — Standard clock enable. Write 1 to gate the clock on for subsystem N.
- `ssN_fast_clk_en` [1] — Fast clock enable. Write 1 to enable the high-speed clock for subsystem N.
- [30:2] — Reserved, write 0.
- `ssN_irq_en` [31] — IRQ enable. Write 1 to route the subsystem N interrupt to the SoC interrupt controller.

`pmod_sel` (offset `0x024`)

- `pmod_ctrl` [7:0] — PMOD routing select. Write the subsystem index (0–3) whose GPIOs should be routed to the PMOD connector. Any value outside 0–3 routes the staff-section GPIOs instead. Reset value `0x4` (staff-section GPIOs active by default).

`io_cell_cfg_N` (offsets `0x028–0x088`, one register per IO pad)

Each register configures one IO pad; only bits [4:0] are connected to the IO cell. Reset value `0x0D` (`5'b01101`).

- `dir` [0] — Pad direction. 0 = output (pad driven from core); 1 = input (pad tristated, value sampled to core).
- [4:1] — Reserved for additional cell parameters (drive strength, slew rate, Schmitt trigger). Not connected in the simulation model.

`return_reg_0` / `return_reg_1` (offsets `0x100` / `0x104`)

- [31:0] — Two consecutive instruction words executed at the CPU program counter after `main()` returns. Both reset to `0x6F` (RISC-V JAL `x0, 0`), so the CPU enters a safe self-loop after program completion.

`boot_reg_0` / `boot_reg_1` (offsets `0x180` / `0x184`)

- [31:0] — Two consecutive instruction words executed at boot before firmware is loaded over JTAG. Both reset to `0x6F` (JAL `x0, 0`), keeping the CPU in a safe idle state until the debugger preloads instruction memory and releases the core.

2.5 Peripherals

Peripheral modules are standard interface modules that interact with external devices. They are reused from PULP Platform modules and are accessible at fixed hardware addresses.

2.5.1 GPIO

GPIOs are general-purpose IO signals that serve no dedicated function. They can be used for a wide variety of purposes, such as reading sensor values or driving an LED. They can interface with any external module that operates at relatively low speed. The GPIOs are exposed on a PMOD connector, enabling connection to standard PMOD modules such as memories or audio devices. Student subsystems that use these pins must implement their own behaviour and synchronization logic internally. The Didactic SoC instantiates 16 GPIO pins (`GPIO[15:0]`); GPIO pad direction is controlled via the `io_cell_cfg` registers in the controller block.

Table 2.6: GPIO register map (base address 0x01030000)

Register name	Offset	Description
REG_PADDIR_00_31	0x00	Pad direction for GPIO[31:0]: 1 = output, 0 = input
REG_GPIOEN_00_31	0x04	GPIO function enable for GPIO[31:0]: 1 enables the GPIO peripheral on that pad
REG_PADIN_00_31	0x08	Sampled input values for GPIO[31:0] (read-only)
REG_PADOUT_00_31	0x0C	Output values for GPIO[31:0]
REG_PADOUTSET_00_31	0x10	Atomic set for GPIO[31:0]: write 1 to drive a pin high; 0 bits unchanged
REG_PADOUTCLR_00_31	0x14	Atomic clear for GPIO[31:0]: write 1 to drive a pin low; 0 bits unchanged
REG_INTEN_00_31	0x18	Interrupt enable for GPIO[31:0]: 1 enables edge detection on that pin
REG_INTTYPE_00_15	0x1C	Interrupt trigger type for GPIO[15:0]: 2 bits per pin
REG_INTTYPE_16_31	0x20	Interrupt trigger type for GPIO[31:16]: 2 bits per pin
REG_INTSTATUS_00_31	0x24	Interrupt status for GPIO[31:0]: set on trigger event, cleared on register read
REG_PADCFG_00_07	0x28	Pad configuration for GPIO[7:0]: 4 bits per pin
REG_PADCFG_08_15	0x2C	Pad configuration for GPIO[15:8]: 4 bits per pin
REG_PADCFG_16_23	0x30	Pad configuration for GPIO[23:16]: 4 bits per pin
REG_PADCFG_24_31	0x34	Pad configuration for GPIO[31:24]: 4 bits per pin
REG_PADDIR_32_63	0x38	Pad direction for GPIO[63:32]
REG_GPIOEN_32_63	0x3C	GPIO function enable for GPIO[63:32]
REG_PADIN_32_63	0x40	Sampled input values for GPIO[63:32] (read-only)
REG_PADOUT_32_63	0x44	Output values for GPIO[63:32]
REG_PADOUTSET_32_63	0x48	Atomic set for GPIO[63:32]
REG_PADOUTCLR_32_63	0x4C	Atomic clear for GPIO[63:32]
REG_INTEN_32_63	0x50	Interrupt enable for GPIO[63:32]
REG_INTTYPE_32_47	0x54	Interrupt trigger type for GPIO[47:32]: 2 bits per pin
REG_INTTYPE_48_63	0x58	Interrupt trigger type for GPIO[63:48]: 2 bits per pin
REG_INTSTATUS_32_63	0x5C	Interrupt status for GPIO[63:32]: set on trigger event, cleared on register read
REG_PADCFG_32_39	0x60	Pad configuration for GPIO[39:32]: 4 bits per pin
REG_PADCFG_40_47	0x64	Pad configuration for GPIO[47:40]: 4 bits per pin
REG_PADCFG_48_55	0x68	Pad configuration for GPIO[55:48]: 4 bits per pin
REG_PADCFG_56_63	0x6C	Pad configuration for GPIO[63:56]: 4 bits per pin

2.5.2 UART

UART is a standardized serial interface typically used to transmit characters to a terminal. The SoC is connected to an FTDI chip which receives data over the UART and forwards it to the host PC over USB. The UART peripheral is a 16450-compatible UART with optional FIFO extensions.

Table 2.7: UART register map (base address 0x01030100)

Register name	Offset	Width	Description
RBR_THR_DLL	0x00	8	Receive Buffer / Transmit Holding / Divisor Latch LSB (multiplexed)
IER_DLM	0x04	8	Interrupt Enable / Divisor Latch MSB (multiplexed)
IIR_FCR	0x08	8	Interrupt ID Register (read) / FIFO Control Register (write)
LCR	0x0C	8	Line Control Register
MCR	0x10	8	Modem Control Register
LSR	0x14	8	Line Status Register (read-only)
MSR	0x18	8	Modem Status Register (read-only)
SCR	0x1C	8	Scratch Register (general-purpose R/W)

2.5.3 SPI

SPI is typically connected to external memories such as an SD card, but can also control any standard SPI device such as sensors. The SPI peripheral supports standard, dual, and quad SPI modes with up to four chip-select lines.

Table 2.8: SPI register map (base address 0x01030200)

Register name	Offset	Description
STATUS	0x00	Transfer mode control and chip-select enable (self-clearing command bits)
CLKDIV	0x04	SPI clock divider
SPICMD	0x08	SPI command word
SPIADR	0x0C	SPI address word
SPILEN	0x10	Transfer lengths: command, address, and data bit counts
SPIDUM	0x14	Dummy cycle counts for read and write phases
TXFIFO	0x18	Transmit FIFO (write-only)
RXFIFO	0x20	Receive FIFO (read-only)
INTCFG	0x24	Interrupt configuration
INTSTA	0x28	Interrupt status (read-clear)

2.6 Student subsystems

Each student subsystem slot provides an experimental area in which a project team can implement a custom subsystem. Access to each subsystem is through an addressable bus. The number of subsystem slots is iteration-specific.

2.6.1 Interface

Subsystems communicate through an addressable bus, APB in this iteration of the SoC. Additional interface signals are provided, carrying configuration and interrupt information. Two of the configuration signals enable the subsystem clocks, while a third enables interrupt propagation out of the subsystem. The remaining wires are available for developer-defined use.

The interrupt signal is routed to the staff core and can be used by the controlling firmware as needed. Each subsystem also has access to the SoC GPIOs, which can be controlled from within the subsystem.

The complete port list of a student subsystem module is shown below. The APB subordinate port connects the subsystem to the SoC interconnect and exposes its internal registers to the CPU. The `ss_ctrl` bus carries the clock-enable and IRQ-enable bits driven by the controller block; bit 0 is the standard clock enable and bit 1 is the fast clock enable. The `irq` output is gated by `irq_en` before being forwarded to the CPU interrupt controller. The two PMOD GPIO ports each provide four bidirectional signals: `gpi` carries sampled input values into the subsystem, `gpo` carries output values driven by the subsystem, and `gpio_oe` is the per-pin output-enable mask.

```
module student_ss_example #(
    parameter APB_AW = 10,
    parameter APB_DW = 32
) (
    // Clock and reset
    input logic          clk_in,
    input logic          reset_int,

    // APB subordinate interface
    input logic [APB_AW-1:0] PADDR,
    input logic          PENABLE,
    input logic          PSEL,
    input logic [APB_DW-1:0] PWDATA,
    input logic          PWRITE,
    input logic [APB_DW/8-1:0] PSTRB,
    output logic [APB_DW-1:0] PRDATA,
    output logic          PREADY,
    output logic          PSLVERR,

    // Subsystem control (from controller block)
    input logic          irq_en_4,
    input logic [7:0]    ss_ctrl_4,

    // Interrupt (to CPU interrupt controller)
    output logic         irq_4,

    // PMOD GPIO port 0 (4-bit bidirectional)
    input logic [3:0]    pmod_0_gpi,
    output logic [3:0]    pmod_0_gpo,
    output logic [3:0]    pmod_0_gpio_oe,

    // PMOD GPIO port 1 (4-bit bidirectional)
    input logic [3:0]    pmod_1_gpi,
    output logic [3:0]    pmod_1_gpo,
    output logic [3:0]    pmod_1_gpio_oe
);
```

2.6.2 Configuration

Initial control sequence:

1. Enable clock for subsystem and ICN
2. Lift ICN and Subsystem reset

3. Access subsystem with hardware memory address

If interrupts or GPIOs are required, they must be enabled through the staff section control registers.

SOC SIMULATION AND VERIFICATION ENVIRONMENT

The Didactic SoC is a system integrating multiple components, including a RISC-V processor core and a debug module which are connected to SRAM memories and APB peripherals through several communication buses. The functional simulation of the Didactic SoC is the first step toward verifying that all modules operate correctly and communicate as expected. This chapter describes the complete simulation flow, from the compilation of test programs to the execution and verification of the SoC behavior.

This simulation is performed using a SystemVerilog testbench and Questa functional simulator from Siemens. The testbench first establishes communication with the Didactic SoC through the JTAG interface, which provides access to the Debug Module Interface (DMI) of the SoC by means of a dedicated shift register in the JTAG Test Access Port (TAP). The debug module is then used to halt the core, configure the SoC, initialize the memories with a software program and set the program counter to the program entry point before execution. Once the core operation is resumed, the execution of the program stimulates the communication paths from the core to memories and peripherals, verifying the correct operation of the SoC in the process. The core finally signals the termination of the program by setting a flag and writing a return status into a debug module register, which can be polled from the testbench through the JTAG interface to confirm that the program completed successfully.

The following sections cover each step of the simulation flow in detail. [Section 3.1](#) presents the software compilation of the test programs provided as part of the Didactic SoC platform, while [Section 3.2](#) addresses the compilation and elaboration of the SoC testbench that instantiates the SoC and peripheral modules. Last but not least, [Section 3.3](#) walks through the simulation process implemented by the testbench.

3.1 Software compilation

The Didactic SoC platform provides several test programs to verify the SoC functionality. Namely, it provides a `blink` test program that stimulates the general purpose input/outputs pins (GPIOs) of the SoC which can be connected to LEDs to visualize the switching activity of the pins, and a `hello` test program that writes "hello from didactic!\n" on the UART, validating the communication with the SoC.

3.1.1 Code organization

Each program consists of a single C source file, complemented by the platform-specific startup file `crt0.S` and linker script `link.ld`, which are automatically included by the build system. The source of the test programs must be located in the `<ROOT_DIR>/sw` folder and organized according to the following convention, where `TESTCASE` is the name of a program:

```
<ROOT_DIR>/  
└─ sw/
```

(continues on next page)

(continued from previous page)

```

├── common/
│   ├── crt0.S
│   ├── link.ld
│   ├── soc_ctrl.h
│   ├── spi.h
│   └── uart.h
├── <TESTCASE>/
│   └── <TESTCASE>.c
└── Makefile

```

3.1.2 Compilation and linking

The programs are compiled using the RISC-V GNU toolchain. The following flags are used to target the RV32IMC instruction set architecture with the ILP32 data model implemented by the Didactic SoC architecture: `-march=rv32imc -mabi=ilp32`.

Note that the runtime environment provided with the platform does not support the C standard library at the moment, and as a result dynamic memory allocation is not available by default. Support for dynamic memory allocation can however be added by providing a custom memory allocator and modifying the linker script to reserve a heap region in the data memory (DMEM).

The build process starts with the compilation of the C and assembly source files into object files:

```

riscv32-unknown-elf-gcc -marchrv32imc -mabi=ilp32 -Icommon/ -c $(TESTCASE)/
↪$(TESTCASE).c -o build/sw/$(TESTCASE).o
riscv32-unknown-elf-gcc -marchrv32imc -mabi=ilp32 -Icommon/ -c common/crt0.S -
↪o build/sw/crt0.o

```

It is followed by a linking step that produces a single ELF binary:

```

riscv32-unknown-elf-gcc -marchrv32imc -mabi=ilp32 -Tcommon/link.ld build/sw/
↪$(TESTCASE).o build/sw/crt0.o -nostartfiles -nostdlib -Wl,--gc-sections -o
↪build/sw/$(TESTCASE).elf

```

The linking step is controlled by the `link.ld` linker script that defines the memory layout expected by the Didactic SoC platform. The script maps the 4 kB instruction memory (IMEM), starting at address `0x01000000`, and the 4 kB data memory (DMEM), starting at address `0x01010000`. The former contains the reset vectors, program instructions and constants, placed respectively in the `.vectors`, `.text` and `.rodata` sections. The latter is divided between a 3 kB stack and 1 kB of statically allocated global data consisting of initialized data in the `.data` and `.sdata` sections, and uninitialized data in the `.bss` section.

The build process ends after converting the ELF binary into a `<TESTCASE>.hex` hex file with little-endian byte ordering and 32-bit word-aligned addresses that is compatible with SystemVerilog's `$readmemh` format using the `elf2hex` utility.

3.1.3 Execution flow

The execution of a test program follows a fixed sequence defined by the startup file `crt0.S`:

```

reset_handler → main → postMain

```

This sequence first initializes the stack and sets the `ra` return address register to the address of the `postMain` cleanup routine before transferring control to the `main` function defined in the C source file

of the test program. The main function then executes and eventually returns a status code in register `a0` to the `postMain` routine. The return value of the main function indicates whether the program succeeded or failed. Upon return from the main function, the `postMain` routine writes the return value into a dedicated debug module register at address `0x01020380` and sets its most significant bit to signal the completion of main. As a result, the end of the program execution can be detected by polling this register over the JTAG interface.

3.1.4 Testbench integration

The build system produces a memory initialization file at `<ROOT_DIR>/build/sw/<TESTCASE>.hex`. The path of the memory image is first passed to the testbench through parameter `TESTCASE` during the elaboration phase. The testbench then reads the image into an internal stimuli array, which is later used as the source for the firmware loading sequence:

```
parameter string HEX_LOCATION = "../build/sw/";
logic [31:0] stimuli [100000:0];
// ...
$readmemh({HEX_LOCATION,TESTCASE,".hex"}, stimuli);
```

3.2 Testbench compilation and simulation

The SoC testbench instantiates the top level of the Didactic SoC along with behavioural models of external peripherals, and defines the stimuli applied to the SoC during simulation. It is compiled and simulated using the Questa functional simulator from Siemens.

Bender is used as a hardware dependency manager to specify external dependencies and define the complete list of source files required by the project. The dependency configuration is defined in the `Bender.yml` file at the root of the project.

Before compiling or simulating the design, third-party IPs must be fetched using the following commands from the project root directory:

```
bender update
bender vendor init
```

The fetched dependencies are stored in the `<ROOT_DIR>/bender/` folder, which is automatically populated and should not be manually modified.

The complete list of source files on which the project depends can then be generated using the following command:

```
bender script flist -t rtl -t vendor -t tracer -t didactic_obi
```

The `-t` flags select the source files associated with specific target groups: `rtl` and `vendor` include the design and third-party IP sources respectively, and `tracer` and `didactic_obi` include additional platform-specific components.

The testbench is compiled and simulated using a Makefile located in the `<ROOT_DIR>/sim/` folder. The flow is organized into three successive targets which can be built individually or in sequence: `compile`, `elaborate` and `run_sim`. The `TESTCASE` variable must be set to the name of the software project to simulate:

```
make compile
make elaborate TESTCASE=blink
make run_sim TESTCASE=blink
```

Each step produces a timestamped log file for debugging purposes in the `<BUILD_DIR>/logs/compile`, `<BUILD_DIR>/logs/vopt` and `<BUILD_DIR>/logs/sim` directories respectively.

3.2.1 Compilation

The `compile` target initializes a Questa simulation library and compiles all RTL and testbench source files into it using `vlog`. The list of source files is generated automatically by Bender, as described previously.

Behavioral models of external peripherals can be selectively instantiated to emulate UART and SPI devices and verify the correct operation of the corresponding interfaces. Their instantiation is controlled by the following parameter definitions in the Makefile:

- `+define+USE_UART`: instantiates the `uart_tb_rx` module, which captures UART characters transmitted by the SoC and writes them to the `stdout/uart` file. When disabled, the UART interface is connected in loopback configuration.
- `+define+USE_SPI`: instantiates the `spi_slave` module, which provides helper tasks to receive and transmit data to the SoC once selected.

3.2.2 Elaboration

The `elaborate` target elaborates and optimizes the compiled design using `vopt`, setting `tb_didactic` as the top-level module. Two parameters are passed to the testbench at this stage:

- `-gTESTCASE=<TESTCASE>`: specifies the name of the software project to simulate, used by the testbench to locate the corresponding hex file in `<ROOT_DIR>/build/sw`.
- `-gDM_SANITY_TESTCASES=<SANITY_CHECK>`: perform debug module sanity checks during simulation when enabled and load the test program in SoC memories through the JTAG interface when disabled. Defaults to `0` (disabled).

3.2.3 Simulation

The `run_sim` target launches the simulation on the elaborated snapshot using `vsim`. It first creates the `stdout/uart` file to capture UART output from the SoC, then starts the simulator. An initial `run 0ms` command is issued to trigger design initialization without advancing simulation time, after which a `run -all` command should be manually issued to drive the simulation autonomously until the program signals its termination through the debug module register.

3.3 Simulation flow

The simulation flow is driven by a sequential process in the testbench which communicates with the Didactic SoC through the JTAG interface. This process follows a pre-defined sequence of operations that can be divided into five phases:

1. reset phase: the SoC is brought out of reset ;
2. JTAG initialization phase: the debug session is established and the core is halted ;
3. Firmware loading phase: the program is written into the instruction memory ;
4. Execution phase: the program counter is set to the entry point and the core is resumed ;

5. Monitoring phase: the testbench polls the SoC for program termination and retrieves the exit status.

3.3.1 Reset sequence

The simulation begins with a reset sequence that brings the SoC into a known state before any JTAG transaction is initiated. The reset signal is first asserted for 3 ms, then released for an additional 3 ms to emulate an external negative reset and the subsequent settling time for the SoC. Only after this period does the testbench begin communicating with the SoC through the JTAG interface.

3.3.2 JTAG interface validation

Before opening a debug session, the testbench performs a series of preliminary checks to validate the JTAG interface. The TAP controller is first brought to a known state using `jtag_reset`, which asserts the TRST signal, followed by `jtag_softreset`, which drives the TAP state machine to the Test-Logic-Reset state by asserting TMS for five clock cycles. The bypass register is then tested using `jtag_bypass_test` to verify that the shift path through the TAP is functional. Finally, the IDCODE register is read using `jtag_get_idcode` and compared against the expected value `0x1C0FFEE1` to confirm that the correct device is connected.

All communication with the debug module is carried out through the Debug Module Interface (DMI), which is accessed via a dedicated shift register (DMIACCESS) in the JTAG TAP controller. Each DMI transaction consists of a register address, a data field and an operation code, which are shifted into the TAP controller's internal shift register on the JTAG clock domain. The transaction is then posted through a clock domain crossing (CDC) circuit to the debug module, which operates on the system clock.

3.3.3 Debug module initialization

Once the JTAG interface is validated, the testbench initializes the debug session. The CONFREG instruction is first selected via the `test_mode_if` interface to configure the SoC. The DMI data register is then selected for subsequent transactions by calling `init_dmi_access`, which loads the DMIACCESS opcode into the JTAG instruction register. The debug module is activated by asserting `dmactive` in the DMControl register via `set_dmactive`, after which the target core is selected using `set_hartsel` and halted using `halt_harts`, which asserts `haltreq` in DMControl. The DMStatus register is polled until the `allhalted` flag is set, confirming that the core has stopped execution. Finally, the boot address is written into the dpc register using `write_reg_abstract_cmd`, setting the program counter to the firmware entry point before resuming the core.

3.3.4 Debug module sanity check

If the `DM_SANITY_TESTCASES` parameter is set to a non-zero value at elaboration time, the testbench runs a series of debug module functional tests instead of loading the firmware. These tests stimulate the core debug module functionalities including core discovery, halt and resume, register access via abstract commands, program buffer execution, single stepping and memory access through the OBI system bus interface. This optional step is intended to verify that the debug module itself operates correctly before relying on it to load and execute a program.

3.3.5 Firmware loading

Once the debug session is established and the core is halted, the firmware is loaded into the instruction memory (IMEM) through the system bus access interface if the `DM_SANITY_TESTCASES` parameter is set to zero at elaboration time (default value). The `load_L2` task iterates over the contents of the hex file generated during the software compilation step and writes each 32-bit word to the target address in IMEM, starting from the base address `0x01000000` and incrementing by four bytes after each write.

3.3.6 Core release and execution

Once the firmware is loaded and the program counter is set to the entry point, control is handed back to the core by calling `resume_harts`, which asserts `resumereq` in `DMControl` and polls `DMStatus` until the `allresumeack` flag is set, confirming that the core has resumed execution. The core then begins executing the firmware from the address stored in the `dpc` register.

3.3.7 Execution monitoring and termination

Once the core is running, the testbench monitors the execution by periodically polling the debug module register at address `0x01020380` every 100 μ s. The termination of the program is detected when bit 31 of the register is set by the `postMain` routine. The exit status is then retrieved from bits 30 to 0 of the same register and reported by the testbench, indicating whether the program completed successfully or encountered an error. The simulation is subsequently stopped.

STUDENT SUBSYSTEM FRONT-END INTEGRATION AND VERIFICATION

All student subsystems share a common interface. This allows for a reusable common testbench design. The basic verification setup can be copied from the provided example and only minor adjustments have to be made in order to adapt the base testbench to the implemented subsystem. The base testbench allows for quick development of constrained random tests for any student subsystem, as the simulation environment, bus interface drivers, and basic test setup are already present and can be reused.

The base testbench is implemented in the Didactic-SoC repository. It contains the reusable base testbench components, as well as some examples on how to implement the DUT specific components and DUT specific testcases.

In the following, it is assumed that the repository root is named `<REPO_ROOT>` and the root of the example test bench is named `<SSTBEX_ROOT>` which is `<REPO_ROOT>/verification/student_ss`. In the provided example the RTL code being tested is the `<REPO_ROOT>/src/rtl/student_ss_example.sv`.

4.1 Student subsystem wrapper

The student testbench relies on a common wrapper module located in `<SSRBEX_ROOT>/common/tb_top/top.sv`. The DUT is instantiated within this module, and the I/O signals are connected to the top module. This ensures the correct connection of the I/O to the testbench signals.

4.1.1 I/O Description

The student subsystems feature an Advanced Peripheral Bus interface (APB), control signals and 2x4 configurable IOs as `pmod_0` and `pmod_1` respectively. The APB Interface has parametrizable address and data widths as described in [Table 4.1](#).

Table 4.1: APB Parameters

Parameter Name	Parameter
APB_AW	APB interface address width
APB_DW	APB interface data width

Table 4.2: APB Signals

Signal name	Direction	Width
PADDR	input	[APB_AW-1:0]
PENABLE	input	1
PSEL	input	1
PWDATA	input	[APB_DW-1:0]
PWRITE	input	1
PSTRB	input	[APB_DW/8-1:0]
PRDATA	output	[APB_DW-1:0]
PREADY	output	1
PSLVERR	output	1

The student subsystem also features eight GPIO pins, clock, reset, interrupt, and control signals as outlined in Table 4.3. The direction of the port is taken from the perspective of the student subsystem.

Table 4.3: Student subsystem general signals

Signal name	Direction	Width
clk_in	Input	1
reset_int	Input	1
irq_4	Output	1
irq_en_4	Input	1
ss_ctrl_4	Input	[7:0]
pmod_0_gpi	Input	[3:0]
pmod_0_gpo	Output	[3:0]
pmod_0_gpio_oe	Output	[3:0]
pmod_1_gpi	Input	[3:0]
pmod_1_gpo	Output	[3:0]
pmod_1_gpio_oe	Output	[3:0]

Student subsystem example

The example testbench implements testcases for a very simple DUT. The example is located in <SSTBEX_ROOT>/src/rtl/student_ss_example.sv.

The DUT implements a very simple APB slave with some added GPIO and control signal functionality. Internally the DUT features three physical registers: `rw_reg`, `gpio_w_reg` and `ss_ctrl_reg`. These registers are mapped on memory addresses `0x00`, `0x08` and `0x0C` respectively, as described in :numref:`table\_toy\_ss\_registers`. The state of the GPIOs is mapped on address `0x04` and can be read by accessing the virtual register `gpio_r_reg`.

Table 4.4: Subsystem example register map and fields

Ad- dress	Register	Description	Bits	Field	Name	Rst.	Access
0x00	rw_reg	32-bit register for reading and writing	[7:0]	field0	field0	0	Read- /Write
			[15:8]	field1	field1	0	Read- /Write
			[31:16]	field2	Unused	0	Read- /Write
0x04	gpio_r_reg	32-bit register for reading of GPIOs	[3:0]	gpio0_fie	GPIO_0	0	Read- /Write
			[7:4]	gpio1_fie	GPIO_1	0	Read- /Write
			[31:8]	unused	Unused	0	Read- /Write
0x08	gpio_w_reg	32-bit register for writing	[3:0]	gpio0_fie	GPIO_0	0	Read- /Write
			[7:4]	gpio1_fie	GPIO_1	0	Read- /Write
			[31:8]	unused	Unused	0	Read- /Write
0x0C	ss_ctrl_reg		[7:0]	ss_ctrl	SS_CTR	0	Read- /Write
			[31:8]	unused	Unused	0	Read- /Write

The content of all registers can be accessed through the APB interface. `rw_reg` can only be accessed through the APB interface. `gpio_r_reg` contains the input data from GPIO 0/1 in the respective field, if they are configured to be inputs. `gpio_w_reg` is written to the GPIO 0/1 pins, if they are configured as outputs. The lower eight bits of the `ss_ctrl_reg` will always contain the input from the `ss_control` signal.

Subsystem simulation using cocotb

The testbench is written in Python using a tool chain using the following three major tools. The RTL design is simulated using [Icarus Verilog](#). [Cocotb](#) is used as a co-simulation tool allowing Python to interact with the simulation. Lastly [pyuvvm](#) implements the Universal Verification Methodology (UVM) framework in Python. The tool stack can be seen in [Fig. 4.1](#)

The installation of the tool stack is documented in `<SSTBEX_ROOT>/readme.md`.

Functional verification primer

The testbench implements the basics to functionally verify the DUT using constrained random verification. The verification is split up into three independent parts:

1. Stimuli generation
2. Response checking
3. Coverage measurement

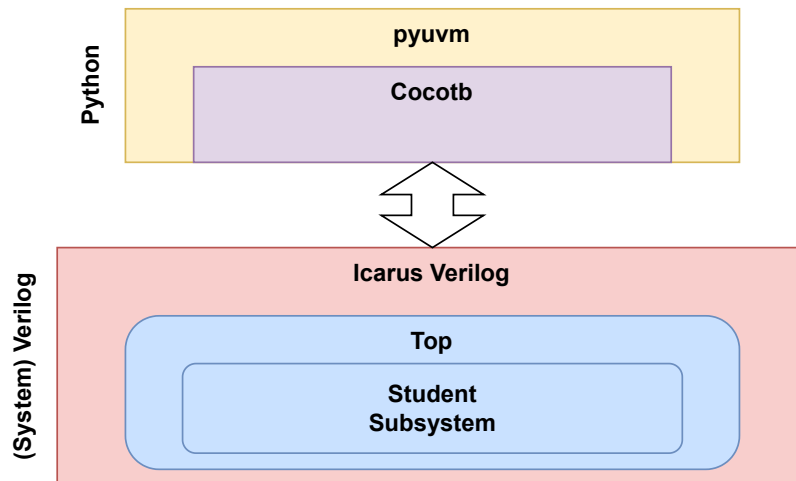


Fig. 4.1: Tool stack used for simulation.

4.1.2 Stimuli generation

Constrained random (stimuli generation) allows for the automatic generation a large number of tests. Instead of writing directed tests, i.e. a set of input stimuli, which exercises a sequence of states in the DUT to test a specific feature, input stimuli are randomly generated. Since truly random stimuli are unlikely to produce interesting tests the generation of stimuli is constrained.

4.1.3 Response checking

Typically the response of the DUT is checked using reference models and assertions. The example testbench has not reference model, but provides an example assertion for the interrupt behavior of the DUT in `<SSTBEX_ROOT>/common/cl_student_tb_assertions.py`.

4.1.4 Coverage measurement

Coverage is the measurement of the verification effort. It measures if all desired states of the DUT have been observed. A good testbench is only as good as the employed coverage model. The testbench also uses the `pyvsc` library for coverage, in addition to randomization.

4.1.5 Transaction level modeling

UVM provides additional abstraction levels for the stimuli generation. Pin level inputs are abstracted using transaction level modeling (TLM). TLM abstracts the pin level signal into individual transactions. A transaction is the minimal description of a bus event. Instead of individually driving the pins of the APB interface transaction can be expressed on a higher abstraction level as a single APB transaction. A APB Transaction, for example, consists of the following fields:

Name	Description
Operation Type	Read/Write Operation
Address	Integer in range $[0, APB_AW*8]$
Data	Integer in range $[0, APB_DW*8]$
Write Strobe	Integer in range $[0, APB_DW]$
Slave Error	Transfer failure

To randomize a transaction, the individual fields of the transaction are randomized, then a driver translates the transaction to the pin level inputs. The testbench typically interacts with the DUT interface through Universal Verification Components (UVC).

Testbench explanation

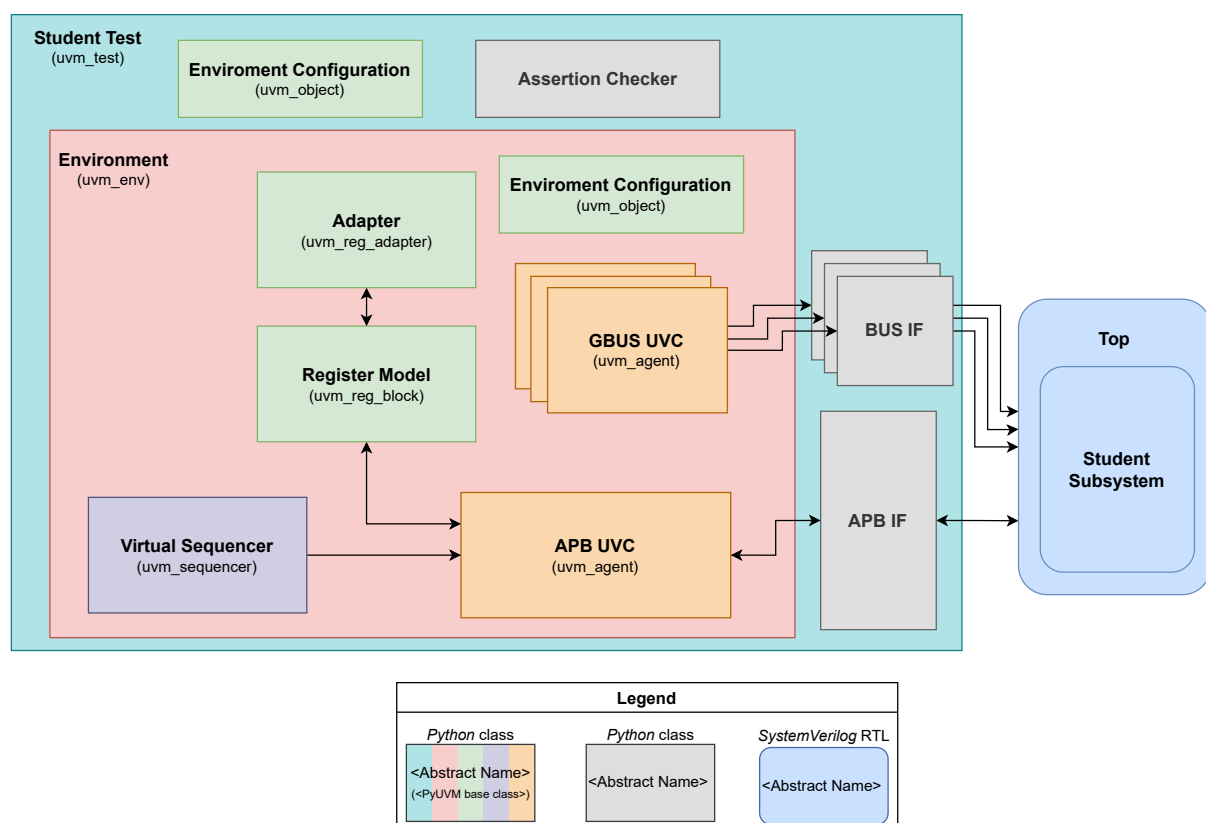


Fig. 4.2: Student subsystem testbench overview

The majority of the student testbench is reusable between different subsystems. In the following sections all parts of the testbench which needs to be configured to adapt the testbench to a new DUT are explained. These parts are highlighted in Fig. 4.2.

UVCs

The testbench has two types of UVCs: the APB UVC, and a general bus (GBUS) uvc. They are located in `<SSTBEX_ROOT>/common/uvc`.

4.1.6 GBUS UVC

The GBUS UVC offers an easy interface to the bus interfaces of the student subsystem. The student subsystem testbench has GBUS UVCs configured for all bus signals. The I/O signals of the student subsystem are grouped into the following busses:

Bus	Signals
PMOD 0 GPI	<code>pmod_0_gpo</code>
PMOD 0 GPO	<code>pmod_0_gpi</code>
PMOD 1 GPI	<code>pmod_1_gpi</code>
PMOD 1 GPO	<code>pmod_1_gpo</code>
SS CTRL	<code>ss_ctrl_4</code>
IRQ EN	<code>irq_en_4</code>
IRQ	<code>irq_4</code>

The respective agents are instantiated in the environment in `<SSTBEX_ROOT>/common/cl_student_tb_env.py`.

The pins of each bus can be driven using the sequences provided in `<SSTBEX_ROOT>/common/uvc/gbus/cl_gbus_seq_lib.py`. Additionally the base sequence in `<SSTBEX_ROOT>/common/cl_student_base_sequence.py` offers two functions `sample_gpo_pins()` and `drive_gpi_pins()` to drive and sample the GPIO pins respectively.

4.1.7 APB UVC

The APB UVC drives and samples the APB interface of the subsystem. It is connected to the register model through the register adapter. The sequences on the UVC are started automatically whenever the register model receives a read or write.

4.1.8 Register Model

The testbench uses a register model for the internal registers of the subsystem. Registers are modeled with the `uvm_reg` and `uvm_reg_block` classes. Each register consists of a number of fields. The registers are grouped together in a register block, which also contains the register map and adapter. Each register field needs to be configured, with the following parameters¹:

1. Field size
2. Least significant bit position
3. Access type
4. Volatility
5. Reset value

There are more optional parameters, see [IEE20] section 18.5 for more optional information. The configuration for the `rw_reg` is shown in [Listing 4.1](#).

¹ The parameters are shown in the order of arguments in the `.configure()` call.

Listing 4.1: UVM Register definition for rw_reg

```

class reg0(uvm_reg):
    """Class : The UVM register class for reg0"""

    def __init__(self, name="reg0", reg_width=32):
        super().__init__(name, reg_width)

        self.field0 = None
        self.field1 = None
        self.field2 = None

    # Build : The build function of reg0.
    def build(self):

        # Constructing all fields in the current register.
        self.field0 = uvm_reg_field("field0")
        self.field1 = uvm_reg_field("field1")
        self.field2 = uvm_reg_field("field2")

        # Configuring all fields of the current register.
        self.field0.configure(self, 8, 0, "RW", True, 0)
        self.field1.configure(self, 8, 8, "RW", True, 0)
        self.field2.configure(self, 16, 16, "RW", False, 0)

```

The individual registers are then grouped together in a `uvm_reg_block`. See `<SSTBEX_ROOT>/common/register_model/student_ss_regs_reg_block.py` for reference.

In the testbench environment, the register model (register block) is connected to the APB UVCs register adapter. The APB UVC then mirrors the read and write accesses to the register model on the DUT.

4.1.9 Writing and reading from registers

To write and read registers, the virtual sequencer in `<SSTBEX_ROOT>/common/cl_student_tb_vsequencer.py` has two functions `reg_write()` and `reg_read()`. `<SSTBEX_ROOT>/common/register_model/cl_reg_dynamic_seq.py` shows how a random read/write sequence can be implemented.²

Adapting the testbench for a new design

To adapt the testbench to a new design, most of the `<SSTBEX_ROOT>/common` folder can be reused.

1. Updating the register model
2. Adapting the virtual sequences
3. Updating the RTL (see `<SSTBEX_ROOT>/readme.md` for more information)

² Note the APB sequence item is only instantiated for easy data generation. The APB UVC is connected through the register adapter and automatically sends APB transaction to the DUT based on the write/read requests to the register model.

4.1.10 Updating the register model

The register model is specific to the example DUT and this needs to be changed to reflect the DUT.

The following files need to be updated:

- The register model (<SSTBEX_ROOT>/common/register_model/student_ss_regs_regs.py)
- The register block (<SSTBEX_ROOT>/common/register_model/student_ss_regs_block.py)
- The dynamic register sequence (<SSTBEX_ROOT>/common/register_model/cl_reg_dynamic_seq.py)
- Subsystem control register sequence (<SSTBEX_ROOT>/common/register_model/cl_reg_ss_ctrl_seq.py)

4.1.11 Adapting the virtual sequences

The virtual sequences in <SSTBEX_ROOT>/common/sequences are kept general such that they can apply to most designs, still they should be checked for applicability to a new design before used, and if necessary adapted.

4.1.12 Writing new testcases

Once the base testbench is adapted to the new subsystem new testcases can be implemented. Under <SSTBEX_ROOT>/tests a few different example testcases are shown.

Any testcase consists of a test-file and virtual sequence (vseq) file. The virtual sequences are placed in <SSTBEX_ROOT>/common/sequences and defines the behavior of the testcase by calling other sequences and starting them.

The test-file should start the sequence on the sequencer and can define assertions for checking results if desired.

SOC INTERFACE AND HW/SW CO-DESIGN

The Didactic SoC developed within the Edu4Chip project is designed to provide you with hands-on experience in SoC integration, combining hardware design with embedded software development. Didactic integrates processors, memories, specific-purpose subsystems, and peripherals into a single chip, requiring efficient communication mechanisms and clear separation between hardware and software layers. The goal of this section is to explain how the firmware is developed in the context of the Didactic SoC and how hardware/software interact with each other.

Developing a firmware for the Didactic SoC involves a structured approach that builds upon the interaction between software and memory-mapped hardware components. The process begins with proper initialization of the platform and the targeted subsystem, followed by the use of abstraction layers to simplify hardware access, and finally the implementation of coordinated hardware/software interaction to perform the desired functionality. This workflow ensures that subsystems can be configured, controlled, and monitored efficiently through standardized interfaces. The following sections detail this process. [Section 5.1](#) describes how the platform and subsystems are initialized. [Section 5.2](#) introduces the Hardware Abstraction Layer (HAL) used to encapsulate low-level register access. [Section 5.3](#) explains how hardware and software are co-designed to enable seamless subsystem integration and operation.

5.1 Platform and Subsystem Initialization

The platform initialization typically is responsible for setting up the essential hardware infrastructure before executing the main application. This process typically includes configuring the system clocks, initializing memory regions, and initializing peripherals and subsystems through a memory-mapped I/O scheme. The processor starts execution from a reset vector stored in ROM, which redirects execution to a reset handler. In general, the reset handler performs several key steps, including initializing the stack pointer, setting up the runtime environment, configuring interrupt handling, and enabling communication buses. In addition to these steps, the correct placement of code and data in memory is ensured by the linker script (`link.ld`), which defines how different program sections (such as `.text` and `.data`) are mapped into the available instruction and data memory regions. The linker script organizes the program into standard sections, where the `.text` section contains executable code, the `.data` section stores initialized global variables, and the `.bss` section holds uninitialized variables that are automatically set to zero at runtime, and the `.stack` section defines the memory region used for function calls and local variables during program execution. It also specifies the location of the stack and important symbols used during program execution, ensuring that the processor accesses instructions and data from the intended memory areas.

The Didactic SoC adopts a simplified configuration only with the essential steps required to start program execution aimed at educational clarity and ease of understanding. As can be seen in `ctr0.S`, the reset vector directly jumps to a `reset_handler`, which initializes the stack pointer and transfers control to the `main` function. This implementation does not include interrupt support, and the vector table contains only basic entries, a default handler and the reset vector. After the main program completes, execution

returns to a dedicated `postMain` routine. It export the result of the program to the outside world through a memory-mapped register accessible via JTAG.

Each subsystem, developed by different partners, is integrated as a memory-mapped IP block. For each subsystem, a predefined memory map specifies the base addresses and register offsets used for control and data exchange. Configuration steps depends on the specific subsystems and typically include setting control registers, enabling or disabling features, and initializing internal states. This is done by writing to registers exposed through the APB interface, making the subsystem behave like a standard peripheral.

The global memory organization of the SoC is summarized in the [Table 5.1](#). It defines the address ranges assigned to instruction memory, data memory, control registers, and each subsystem slot. In particular, student subsystems are mapped within the interconnect region starting at address `0x0105_0000`, with each subsystem occupying a dedicated address range. By referring to this memory map, firmware developers can determine the correct base addresses and offsets required to access and configure each subsystem during initialization.

Table 5.1: Memory map

Memory map target	Start	End	Actual size
Instruction memory	0x0100_0000	0x0100_FFFF	16 KiB
Data memory	0x0101_0000	0x0101_FFFF	16 KiB
Debug module	0x0102_0000	0x0102_FFFF	0x900
Staff peripherals	0x0103_0000	0x0103_FFFF	0x300
Control registers	0x0104_0000	0x0104_FFFF	0x184
Interconnect	0x0105_0000	0x0105_FFFF	Denoted by number of ss
Student 0	0x0105_0000	0x0105_0FFF	
Student 1	0x0105_1000	0x0105_1FFF	
Student 2	0x0105_2000	0x0105_2FFF	Template is empty
Student 3	0x0105_3000	0x0105_3FFF	Template is empty

5.2 Hardware Abstraction Layer

The Hardware Abstraction Layer (HAL) provides a structured interface to configure and interact with hardware components within the SoC. Its primary purpose is to isolate low-level register manipulations from application-level software, enabling developers to interact with peripherals through well-defined and reusable APIs. The HAL acts as a wrapper around low-level drivers, enabling portability and maintainability across platform and subsystems.

In the Didactic SoC, the HAL is designed using a modular approach, where each peripheral or subsystem is encapsulated within a dedicated header file. Each module typically provides Register definitions (base addresses and offsets), Macros for memory-mapped access, a set of control functions that directly manipulate memory-mapped control registers.

The common HAL modules include:

- Control HAL (`soc_ctrl.h`): global SoC and subsystem control
- GPIO HAL (`gpio.h`): Provides functions to configure pins as input/output and read/write digital values.
- UART HAL (`uart.h`): Enables serial communication through functions for initialization, transmission, and reception.
- SPI HAL (`spi.h`): Supports communication with external devices using the SPI protocol.

5.2.1 Control HAL (`soc_ctrl.h`)

The Control HAL provides low-level access to global SoC configuration registers responsible for managing subsystem reset, clock control, and external routing. It is based on a set of memory-mapped registers defined relative to a control base address (`CTRL_BASE`), where each subsystem is associated with a dedicated control register. It provides several abstraction functions. Functions such as `ss_init()` and `ss_init_high_speed()` enable a target subsystem by configuring its reset state and activating its clock, while `ss_reset()` disables the subsystem and clears its configuration. In addition, the `pmod_target()` function allows routing of external I/O (PMOD) signals to a selected subsystem.

This HAL encapsulates bit-level manipulation of control registers, allowing software to initialize and manage subsystems through simple function calls rather than direct register access. It plays a central role in platform initialization and subsystem activation within the Didactic SoC.

5.2.2 GPIO HAL (`gpio.h`)

The GPIO HAL provides an interface for configuring and controlling general-purpose input/output pins. It defines memory-mapped registers for reading input values (`PAD_IN`) and writing output values (`PAD_OUT`), as well as a set of configuration registers used to control the direction and behavior of each GPIO pin. Functions such as `gpio_init_out()` and `gpio_init_in()` configure individual pins as outputs or inputs by updating the corresponding pad configuration registers.

Once configured, the GPIO pins can be accessed through `gpio_write()` to drive output values and `gpio_read()` to sample input signals. This HAL enables straightforward interaction with external hardware components while abstracting the underlying register-level operations.

5.2.3 UART HAL (`uart.h`)

The UART HAL provides a minimal interface for serial communication through a set of memory-mapped registers controlling transmission, reception, and configuration. The `uart_init()` function configures the UART by setting the baud rate divisor, enabling FIFO buffers, and initializing control registers for standard communication. It also configures the corresponding I/O pad for reception.

Data transmission is handled by the `uart_print()` function, which sends characters sequentially by writing to the transmit register. Due to the absence of interrupt-based handling in this implementation, a simple delay loop is used to ensure correct timing between transmissions. Additionally, a basic `uart_loopback_test()` function is provided to verify functionality by transmitting and reading back a character.

5.2.4 SPI HAL (`spi.h`)

The SPI HAL provides functionality for communicating with external devices using the Serial Peripheral Interface protocol. It defines a set of memory-mapped registers for controlling SPI transactions, including command configuration, data length, address, and FIFO buffers for data transfer. The `spi_init()` function configures the pad settings for SPI signals, preparing the interface for communication.

The HAL supports both read and write operations through `spi_read()` and `spi_write()` functions. These functions configure the SPI transaction parameters, set the appropriate command type, and control the direction of the data pins by updating pad configuration registers. Data is transferred through transmit and receive FIFOs, while control and status registers manage the execution of the transaction.